

PROJECT REPORT

AI Medical Report Assistant

*An End-to-End Deep Learning and Generative AI System for Chest X-Ray Pneumonia
Classification and Automated Reporting*

Submitted By

Govardhan Sunkari

B.Tech — Artificial Intelligence & Machine Learning

GitHub: github.com/Govardhan-17/AI-Medical-Report-Assistant

July 2026

Table of Contents

1. Introduction	3
1.1 Problem Statement	3
1.2 Project Objectives	3
2. Technologies Used	3
2.1 Project Structure	4
3. Dataset	4
3.1 Dataset Statistics	5
3.2 Label Distribution & Sample Images	5
4. Deep Learning Model Architecture	5
4.1 Training & Evaluation Pipeline	6
4.2 Evaluation Metrics Tracked	6
5. Model Performance & Results	6
5.1 Interpretation	6
5.2 Generated Evaluation Artifacts	7
6. AI-Assisted Medical Report Generation	8
6.1 Report Contents	8
6.2 Design Rationale	9
7. Database Design (SQLite)	9
7.1 Stored Fields	9
8. System Architecture & Application Workflow	9
9. Installation & Setup	12
9.1 Clone the Repository	12
9.2 Create and Activate a Virtual Environment	12
9.3 Install Dependencies	12
9.4 Configure Environment Variables	12
9.5 Train and Evaluate the Model	12
9.6 Run the Application	12
10. Future Improvements	12
10.1 Conclusion	13
11. Disclaimer	13
12. References & Author	13
12.1 References	13
12.2 Author	13

1. Introduction

The AI Medical Report Assistant is an end-to-end artificial intelligence application that analyzes chest X-ray images and classifies them as NORMAL or PNEUMONIA. The system combines a deep learning computer-vision model with a large language model (LLM) to move beyond a simple label prediction: once the image is classified, the application uses Google Gemini to automatically draft a structured, human-readable medical report that summarizes the finding, states a confidence score, and outlines general precautions — all clearly framed as AI-assisted output rather than a clinical diagnosis.

The project brings together several layers of a modern applied-AI stack in a single working system: transfer learning for image classification, a generative-AI layer for natural-language reporting, a lightweight database for persistence, and an interactive web interface for end users. It was built as an educational and portfolio project to demonstrate practical, end-to-end delivery of an AI product — from raw dataset to a usable application — rather than an isolated modelling exercise.

1.1 Problem Statement

Radiological diagnosis of pneumonia from chest X-rays is a task that requires trained expertise and can be time-consuming, particularly in settings with a high patient load or limited access to specialists. An automated first-pass triage tool that can flag likely pneumonia cases and present findings in plain language can help prioritize cases for review, support training and awareness, and illustrate how computer vision and generative AI can be combined in a healthcare-adjacent workflow — while making unambiguously clear that it does not replace a qualified radiologist or physician.

1.2 Project Objectives

- Build an accurate binary image classifier that distinguishes NORMAL from PNEUMONIA chest X-rays using transfer learning.
- Integrate a generative language model (Google Gemini) to translate a raw prediction and confidence score into a structured, readable AI-assisted medical report.
- Provide an accessible, interactive web interface (Streamlit) so a non-technical user can upload an image and receive a result immediately.
- Persist every prediction, along with its AI-generated report, in a local database for historical tracking and audit.
- Evaluate the trained model rigorously using standard classification metrics and visualizations (accuracy, precision, recall, F1, ROC/AUC, confusion matrix).
- Clearly communicate the educational nature of the tool through an explicit medical disclaimer at every stage of the workflow.

2. Technologies Used

Layer	Technology	Purpose
Language	Python 3.12+	Core implementation language
Deep Learning	TensorFlow / Keras	Model training and inference
Model Backbone	EfficientNetB0 (Transfer Learning)	Chest X-ray image classification

Layer	Technology	Purpose
Generative AI	Google Gemini API	Automated AI-assisted medical report generation
Web Interface	Streamlit	Interactive front end for upload and results
Database	SQLite	Storing prediction history and reports
Image Processing	OpenCV	Image loading and preprocessing
Data Handling	NumPy, Pandas	Array operations and tabular data / CSV export
Visualization	Matplotlib	Accuracy, loss, AUC and ROC curve plots
Evaluation	Scikit-learn	Metrics: precision, recall, F1, confusion matrix, ROC/AUC

2.1 Project Structure

The codebase is organized to separate the model training pipeline from the Streamlit application logic:

```

AI-Medical-Report-Assistant/
├── app/
│   ├── __init__.py
│   ├── main.py           (Streamlit entry point)
│   ├── predict.py       (image preprocessing + inference)
│   ├── database.py      (SQLite read/write)
│   ├── llm.py           (Gemini report generation)
│   └── utils.py
├── database/             (SQLite database file)
├── dataset/
│   ├── train/
│   ├── val/
│   └── test/             (evaluation plots)
├── images/              (saved trained model)
├── model/               (user-uploaded X-rays)
├── uploads/
├── train.py             (model training script)
├── evaluate.py          (evaluation & plot generation)
├── requirements.txt
├── README.md
└── .gitignore

```

3. Dataset

The model is trained on the publicly available Chest X-Ray Images (Pneumonia) dataset, originally curated by Paul Mooney and hosted on Kaggle. The dataset consists of anonymized, labeled pediatric chest radiographs organized into training, validation, and test splits.

Source: <https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia>

3.1 Dataset Statistics

Split	NORMAL	PNEUMONIA	Total
Train	1,341	3,875	5,216
Validation	8	8	16
Test	234	390	624
Overall	1,341 (25.7%)	3,875 (74.3%)	5,856

The class distribution is notably imbalanced, with pneumonia cases making up roughly three-quarters of the training data. This imbalance is an important factor in interpreting the model's precision/recall trade-off discussed in Section 5.

3.2 Label Distribution & Sample Images

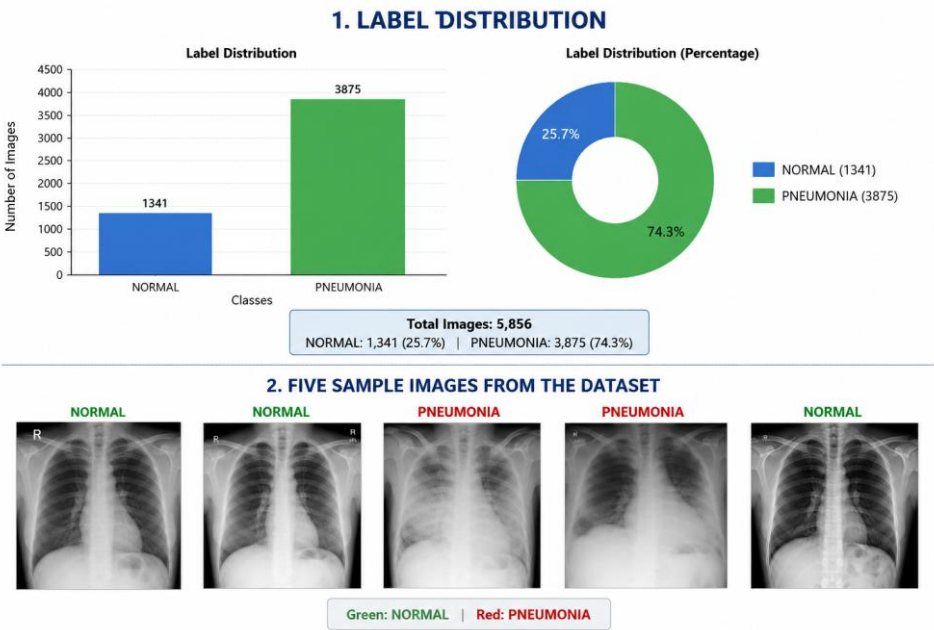


Figure 1 — Left: class balance across the full dataset (1,341 NORMAL vs. 3,875 PNEUMONIA images). Right: five representative chest X-ray samples from the dataset, labeled by ground-truth class.

4. Deep Learning Model Architecture

The classifier is built using transfer learning on top of EfficientNetB0, a convolutional neural network pretrained on ImageNet. Transfer learning allows the model to reuse general-purpose visual features learned from millions of natural images and fine-tune them for the specific task of distinguishing normal lungs from pneumonia-affected lungs, which is especially effective given the relatively modest size of the medical imaging dataset.

Parameter	Value
Base Architecture	EfficientNetB0 (ImageNet pretrained)
Input Image Size	224 × 224
Optimizer	Adam
Loss Function	Binary Crossentropy

Parameter	Value
Task Type	Binary Classification (NORMAL vs. PNEUMONIA)

4.1 Training & Evaluation Pipeline

- train.py — loads the train/validation splits, applies preprocessing, and fine-tunes EfficientNetB0 on the labeled chest X-ray images, saving the trained weights to the model/ directory.
- evaluate.py — runs the trained model against the held-out test split and produces the accuracy, loss, AUC, ROC curve, and confusion matrix visualizations used for reporting.

4.2 Evaluation Metrics Tracked

- Accuracy
- Precision
- Recall
- F1 Score
- ROC Curve and AUC (overall and validation)
- Confusion Matrix
- Full classification report

5. Model Performance & Results

On the held-out test set of 624 images, the fine-tuned EfficientNetB0 model achieved the following results:

Metric	Result
Accuracy	89.10%
Precision	86.76%
Recall	97.44%
F1 Score	91.79%
Test AUC	97.28%

The model achieved strong recall (97.44%), making it suitable as a screening aid for detecting pneumonia while minimizing missed positive cases.

5.1 Interpretation

The model's recall of 97.44% indicates it correctly identifies the large majority of true pneumonia cases, which is the more clinically important error mode to minimize in a screening context — missing an actual pneumonia case (a false negative) is generally more costly than flagging a normal X-ray for a second look (a false positive). The precision of 86.76%, somewhat lower than recall, reflects a tendency toward over-flagging positives, which is a reasonable and arguably intentional trade-off for a triage/screening tool rather than an autonomous diagnostic system. The high test AUC of 97.28% shows strong overall separability between the two classes across the range of decision thresholds, and the F1 score of 91.79% confirms a well-balanced precision/recall trade-off despite the class imbalance in the training data (Section 3.1).

5.2 Generated Evaluation Artifacts

Running `evaluate.py` produces the following visual diagnostics, saved to the `images/` directory for inclusion in reporting and documentation:

- Accuracy curve (training vs. validation, across epochs)
- Loss curve (training vs. validation, across epochs)
- AUC curve (training vs. validation, across epochs)
- Confusion matrix (test set predictions vs. ground truth)
- ROC curve (true positive rate vs. false positive rate)

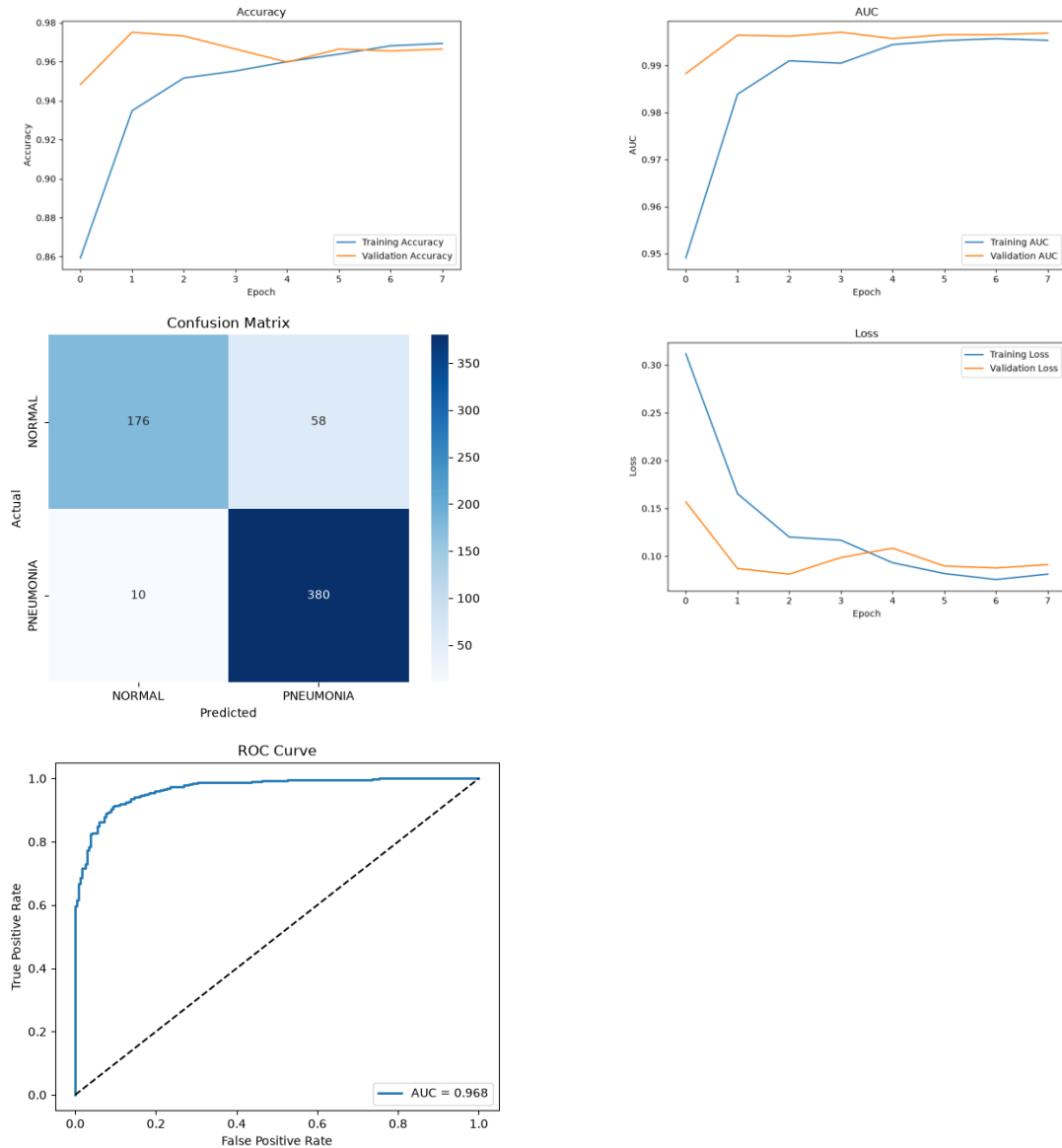


Figure 2 - Evaluation artifacts generated by `evaluate.py`: accuracy, loss, AUC, ROC curve, and confusion matrix.

6. AI-Assisted Medical Report Generation

A key differentiator of this project is that it does not stop at a raw classification label. After EfficientNetB0 produces a prediction and confidence score, that structured output is passed to Google Gemini, which synthesizes it into a plain-language, patient-friendly report. This bridges the gap between a numeric model output and something a non-specialist end user can actually read and understand.

6.1 Report Contents

- Disease prediction (NORMAL or PNEUMONIA)
- Model confidence score
- Possible radiological findings consistent with the prediction
- Possible associated symptoms
- General precautions and next steps

- An explicit recommendation to consult a qualified healthcare professional
- A medical disclaimer restating the educational, non-diagnostic nature of the output

6.2 Design Rationale

Framing the LLM's role narrowly — as a communicator of an existing model prediction rather than an independent diagnostician — keeps the system's two AI components appropriately scoped: EfficientNetB0 performs the actual image classification, while Gemini's role is limited to language generation around that already-computed result. The recommendation to consult a healthcare professional and the medical disclaimer are treated as mandatory, non-optional fields in every generated report.

7. Database Design (SQLite)

Every prediction made through the application is persisted locally using SQLite, giving the system a lightweight audit trail without the operational overhead of a client-server database.

7.1 Stored Fields

Field	Description
Image Name	Filename of the uploaded chest X-ray
Prediction	NORMAL or PNEUMONIA
Confidence Score	Model's prediction confidence (%)
AI-Generated Medical Report	Full Gemini-generated report text
Timestamp	Date and time of the prediction

The Streamlit interface exposes this history directly to the user, who can review past predictions and export the full history as a CSV file for external analysis or record-keeping.

8. System Architecture & Application Workflow

End to end, a single prediction flows through the system as shown below and summarized in Table 8.1.

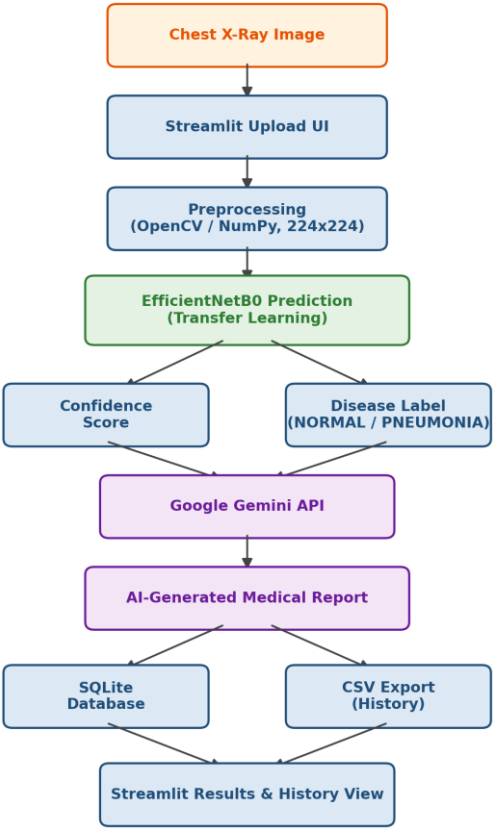


Figure — End-to-end application architecture and data flow

Figure 2 — End-to-end application architecture and data flow, from image upload through prediction, AI report generation, and persistence.

Step	Stage	Description
1	Upload	User uploads a chest X-ray image through the Streamlit interface.
2	Preprocessing	The image is resized to 224×224 and normalized using OpenCV / NumPy.
3	Prediction	EfficientNetB0 classifies the image as NORMAL or PNEUMONIA.
4	Confidence Score	The model's prediction confidence is computed and displayed.
5	AI Report Generation	Google Gemini generates a structured, human-readable medical report.
6	Persistence	The image name, prediction, confidence, report, and timestamp are saved to SQLite.
7	History Display	Past predictions are shown in the app, retrievable at any time.
8	Export	The full prediction history can be downloaded as a CSV file.

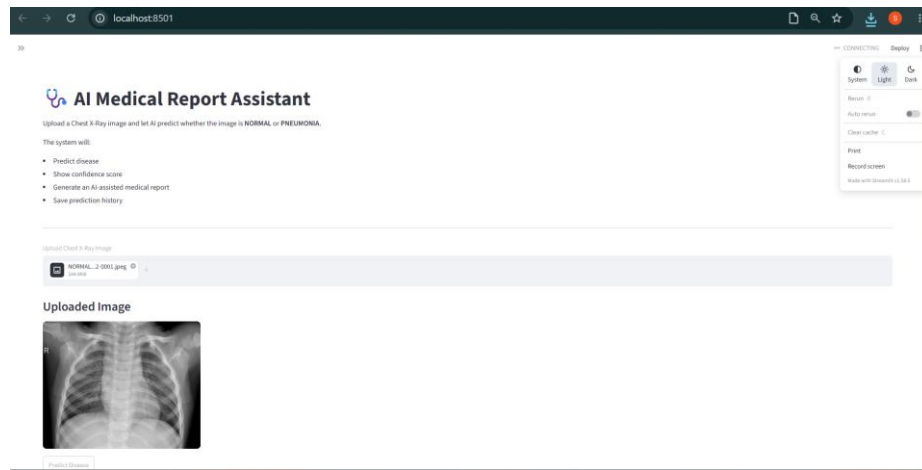


Figure 3 - Streamlit application: upload and prediction result view.

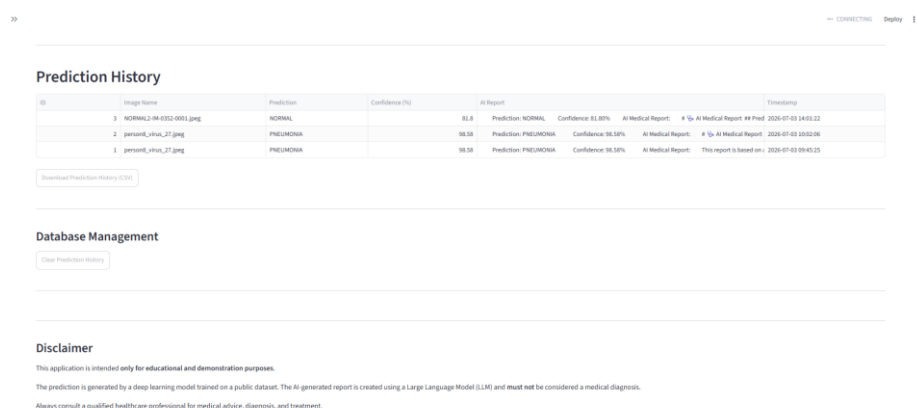


Figure 4 - Streamlit application: prediction history view.

9. Installation & Setup

9.1 Clone the Repository

```
git clone https://github.com/Govardhan-17/AI-Medical-Report-Assistant.git
cd AI-Medical-Report-Assistant
```

9.2 Create and Activate a Virtual Environment

```
python -m venv venv
# Windows
venv\Scripts\activate
```

9.3 Install Dependencies

```
pip install -r requirements.txt
```

9.4 Configure Environment Variables

Create a `.env` file in the project root containing the Gemini API key (obtainable from aistudio.google.com/app/apikey):

```
GEMINI_API_KEY=YOUR_GEMINI_API_KEY
```

9.5 Train and Evaluate the Model

```
python train.py
python evaluate.py
```

9.6 Run the Application

```
streamlit run app/main.py
```

10. Future Improvements

- Multi-class disease classification — extending beyond the binary NORMAL/PNEUMONIA task to detect additional thoracic conditions.
- Grad-CAM visualization — overlaying class-activation heatmaps on X-rays to make model predictions more interpretable and clinically trustworthy.
- PDF medical report export — allowing users to download the AI-generated report directly as a formatted PDF, alongside the current in-app view.
- Docker support — containerizing the application for consistent, portable deployment.
- Cloud deployment — hosting the application on AWS, Azure, or GCP for public accessibility.
- User authentication — adding login/role-based access for multi-user scenarios.
- Patient dashboard — a dedicated view for tracking an individual patient's prediction history over time.

- REST API support — exposing prediction and report-generation functionality as an API for integration with other systems.

10.1 Conclusion

The AI Medical Report Assistant successfully combines deep learning, transfer learning, generative AI, and an interactive web application into a complete end-to-end medical image analysis system. The EfficientNetB0 model achieved an accuracy of 89.10% and a test AUC of 97.28%, while Google Gemini generated human-readable medical reports from model predictions. The system also provides prediction history management using SQLite and CSV-based export. Although intended solely for educational purposes, the project demonstrates how modern AI technologies can be integrated to build intelligent healthcare support applications.

11. Disclaimer

This project was developed solely for educational and research purposes. The predictions generated by the deep learning model and the AI-generated medical report must not be considered a medical diagnosis. Users should always consult a qualified healthcare professional for actual diagnosis, treatment, and medical advice. The authors and contributors accept no liability for decisions made based on the output of this application.

12. References & Author

12.1 References

- Dataset — Chest X-Ray Images (Pneumonia), Paul Mooney, Kaggle: <https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia>
- EfficientNet — Tan, M. & Le, Q. (2019). "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks."
- TensorFlow / Keras documentation — <https://www.tensorflow.org/>
- Keras documentation — <https://keras.io/>
- Streamlit documentation — <https://docs.streamlit.io/>
- Google Gemini API documentation — <https://ai.google.dev/>
- Scikit-learn documentation — <https://scikit-learn.org/>

12.2 Author

Govardhan Sunkari

B.Tech — Artificial Intelligence & Machine Learning

GitHub: github.com/Govardhan-17

Project Repository: github.com/Govardhan-17/AI-Medical-Report-Assistant